CS543 | WEB SERVICES | PROJECT SUBMISSION

SecureCloud

Detailed objectives, implementation and evidence for Assignments 1-6.

> **Project:** SecureCloud private cloud-storage application
> **Course:** CS543 Web Services
> **Submission:** Assignments 1-6

Team members

Roll number	Full name
20252651051	Shirish Meshram
20252651017	Deepamoni Boro
20252651006	Aman Chouhan
20252651048	Sanu Sawarnakar

Project repository

https://github.com/si-rish/SecureCloud

Working demonstration

> **Google Drive demo link:**
> [ADD SHAREABLE GOOGLE DRIVE LINK HERE]
>
> Upload the working demonstration to Google Drive, set access to **Anyone with the link - Viewer**, and replace this placeholder before submission.

How the six assignments fit together

The assignments build from understanding HTTP, to designing services and a SOAP integration, and then implementing and hardening an HTTP API. We applied the coursework to SecureCloud's existing React and Node.js/Express project so the explanations and working examples connect to one application.

> **Scope:** The coursework examples include CampusEats/Orders and Python. This submission adapts the same web-service learning objectives to SecureCloud's File Service. The SOAP scanner is a design example using reserved example addresses; it is not a live third-party service.

ASSIGNMENT 1 | HTTP AND PROJECT SETUP

Understand the application and HTTP

Objective

Understand what the project does, identify its users and main actions, and learn how a web client communicates with a service. Examine real request methods, URLs, status codes, headers and response bodies, then inspect browser traffic for the project page.

How we achieved it

1. **Defined the project brief.** We described SecureCloud as private cloud storage for organization members and administrators. We identified users, files, folders, shares and permissions as important concepts, and listed actions such as sign-in, organizing, sharing, deleting and restoring files.
2. **Recorded real HTTP exchanges.** We used `curl -i` with JSONPlaceholder GET endpoints for posts, albums and todos, plus a missing post. The log includes request lines and headers, response status and headers, and example JSON bodies, including a real 404.
3. **Inspected the browser Network panel.** We loaded the local SecureCloud login page with caching disabled and recorded request count, transferred data, response codes and slowest resource. The write-up explains that this is a Vite development build, not a production-size measurement.

What this teaches

A URL identifies a resource; the method describes the requested operation; headers express metadata and preferences; the status communicates the outcome; and the response body carries the returned representation or error. Browser developer tools expose these exchanges for debugging.

> **Evidence:** `docs/assignments/assignment-01/brief.md`, `http-log.md` and `network-analysis.md`.

ASSIGNMENT 2 | SERVICES, CONTRACTS AND SCHEMA

Design clear service boundaries

Objective

Divide the application into services with focused responsibilities, describe how they cooperate through contracts, define what information each owns, and sketch a database schema without making services share private tables.

How we achieved it

1. **Defined three responsibilities.** Identity owns user accounts and account status; File owns folders and file metadata/version references; Sharing owns share records, recipients, permissions and expiry.
2. **Described contract operations.** The design lists example inputs, outputs and failures for registering/authenticating users, checking account access, managing files, and creating, retrieving or revoking a share.
3. **Specified service-to-service calls.** A file operation asks Identity whether the owner is active. Sharing asks File for ownership and Identity for recipient status. These are contract calls; one service must not query another service's database directly.
4. **Designed service-owned data.** The ER/schema material documents users, files/folders and shares. Cross-service user/file references remain identifiers rather than foreign keys into another service's tables.
5. **Explained the service properties.** The design notes reachability, contracts, independence and coupling, with practical mitigations such as timeouts and clear dependency errors.

How this relates to the implementation

This assignment is the design target, not a claim that three independent production deployments already exist. The later coursework implements the File Service and a local Identity demo as separate Express applications that communicate over HTTP. The Sharing Service remains a documented boundary.

> **Evidence:** `docs/assignments/assignment-02/design.md`, the editable and PNG service/schema diagrams, and `database/schemas/assignment-02-schema.sql`.

ASSIGNMENT 3 | EXTERNAL SOAP INTEGRATION

Specify a partner contract

Objective

Explain how SecureCloud could integrate with an external web service that requires SOAP. Define the messages and operation in a WSDL, describe how to find/bind to the partner contract, and map partner faults into safe application errors.

How we achieved it

1. **Selected a realistic boundary.** The File Service needs a malware scanner before a file can be trusted. SOAP is used only at this external boundary; the rest of the application can continue using REST.
2. **Defined a SOAP 1.1 operation.** The WSDL specifies `ScanFile`, its XML input/output types, SOAP messages, port type and HTTP binding. The example request carries a file reference, media type, checksum and short-lived download URL, rather than the internal storage key or user credentials.
3. **Documented discovery and transport.** The write-up shows the contract/endpoint catalogue fields and an HTTPS POST with the content type and SOAPAction expected by the contract.
4. **Mapped faults to our own errors.** A threat-detected SOAP fault maps to an invalid-file result; partner XML, credentials, storage keys and stack traces are not returned to a SecureCloud caller.
5. **Kept the example honest.** The partner hostname is under reserved `.example.com`; the WSDL and SOAP request/response/fault are illustrative documents, not a live partner or running scanner.

> **Evidence:** `docs/assignments/assignment-03/integration.md`, `partner.wsdl`, request/response/fault XML, printable integration PDF and submission ZIP.

ASSIGNMENT 4 | IMPLEMENT A REST FILE SERVICE

Build a working HTTP API

Objective

Implement a service that exposes resources through HTTP, validates input, returns appropriate statuses and a documented contract, interacts with the Identity boundary, and provides automated evidence that expected cases work.

How we achieved it

1. **Built on the repository's stack.** We used the existing Node.js/Express backend rather than introducing the worksheet's separate Python sample application.
2. **Implemented metadata operations.** `POST /api/files` registers metadata; GET reads one file or lists files by owner; PATCH changes permitted metadata; DELETE marks a file deleted; and a restore action restores a deleted file. This assignment does not upload or store file bytes.
3. **Defined public data and validation.** The API validates UUIDs, file name, media type, size, checksum and allowed fields. It returns public file metadata but not the internal storage key.
4. **Integrated Identity through HTTP.** Before creation, the File Service calls the Identity Service's access-status endpoint and creates only for an active owner. A local demo Identity application makes this dependency runnable without another system.
5. **Documented and tested the API.** An OpenAPI contract describes request/response schemas and errors. Node's built-in test runner verifies creation, reads, listing, invalid requests and dependency outcomes.

> **Evidence:** `backend/src/files/`, `backend/src/identity-demo.js`, `backend/openapi.yaml`, tests and Assignment 4 curl transcript.

Storage note: File metadata and idempotency records are held in memory for the coursework demo and reset on restart. File content is not stored.

ASSIGNMENT 5 | HTTP METHODS, HEADERS AND RETRIES

Make HTTP behavior explicit

Objective

Use methods and status codes correctly; support negotiation, conditional requests, authorization signaling, rate limiting and browser security; and retry only operations that are safe to retry.

How we achieved it

- **Methods and resource paths:** list/read use GET, creation uses POST, partial metadata change uses PATCH, deletion uses DELETE, and OPTIONS exposes allowed methods. Unsupported operations return 405 with `Allow`. Method override is documented as a constrained-client fallback.
- **Safe retry of creates:** POST requires an `Idempotency-Key`. Repeating the same key/body returns the original creation; using a key for different data returns 409.
- **Conditional caching and edits:** a file GET returns an ETag and matching `If-None-Match` receives bodyless 304. PATCH requires the current `If-Match`; stale edits receive 412 and missing preconditions 428.
- **Request/response headers:** missing Authorization header receives 401 and `WWW-Authenticate`. JSON Accept negotiation, CORS preflight, Helmet security headers and gzip are covered.
- **Rate limiting:** the in-memory per-IP limiter returns 429 with `Retry-After` when the client budget is exhausted.
- **Dependency retries:** Identity checks use a timeout and limited backoff retries only for network failures, 408, 429 and 5xx; non-retryable 4xx responses are not retried. If Identity remains unavailable, create fails closed with 503.

> **Evidence:** eight assignment answers, method/header tables, retry plan, OpenAPI updates, 19-test milestone results and live Assignment 5 curl transcript are included in backend notes and the Assignment 5 ZIP. The final Assignment 6 test run rechecks these behaviors.

Demo limitations: Authorization checks only for a non-empty header and does not validate credentials. The rate limiter is process-local; both need production-grade replacements for a real deployment.

ASSIGNMENT 6 | HONEST ERRORS, VALIDATION AND FORMATS

Return useful, consistent results

Objective

Ensure HTTP statuses accurately indicate outcomes, standardize failures as Problem Details, collect field-level validation errors before doing work, negotiate response formats, and prove success/failure behavior with live requests while retaining Assignment 5 retry and idempotency behavior.

How we achieved it

- **Status is the result:** creation returns 201 and Location; successful reads/updates/restores return 200; delete returns bodyless 204; missing resources return 404; invalid data returns 422; stale ETags return 412; conflicts return 409; rate limiting and unavailable dependencies return 429/503.
- **One error shape:** errors use `application/problem+json` with stable `type`, `title`, `status` and safe `detail`. Unexpected errors are logged server-side and the caller gets a generic response, not internals.
- **Field-level validation:** create validates UUID, file name, media type, non-negative size, checksum and optional folder. The server collects every invalid field into an `errors[]` list in one 422 response before contacting Identity or changing state. PATCH also collects metadata errors.
- **Content negotiation:** JSON is the normal representation. `Accept: application/xml` returns XML; unsupported `Accept` returns 406. Responses declare UTF-8 content types, and JSON/XML validators are distinct.
- **Retry guidance:** 429 and 503 include `Retry-After`. Safe retry rules for GET, idempotency-key creates, conditional PATCH and action POST are explained in NOTES.md.
- **Proof and regression checks:** the curl transcript includes create/replay, JSON and XML, 304, successful/stale/missing conditional updates, multi-field 422, 400/401/404/405/406/409, 429, 503 and delete. Automated tests also simulate Identity timeouts and retries.

> **Verification:** 23 automated tests passed. Redocly validates the OpenAPI contract with zero errors; it reports one warning because no project license was supplied.

Evidence: Assignment 6 answers and status map in `backend/NOTES.md`, updated `backend/openapi.yaml`, `backend/transcript.txt`, test output and `docs/assignments/assignment-06-submission.zip`.

RUN GUIDE | FINAL SUBMISSION CHECKLIST

Run, verify and submit

Run locally

From `backend/`, install packages if needed and start the local Identity demo:

```
npm install
npm run start:identity-demo
```

In a second terminal, start the File Service:

```
npm start
```

The demo recognizes owner `8c71c880-fcb8-4ea8-a4e1-10c2118bd864`. In a third terminal, run the tests:

```
npm test
```

Main deliverables

- Assignment 1-3 materials in `docs/assignments/assignment-01/` through `assignment-03/`.
- Assignment 4-6 implementation, tests and OpenAPI contract under `backend/`.
- Assignment 5 and Assignment 6 submission ZIPs in `docs/assignments/`.
- Detailed status, error catalogue, retry explanations and answers in `backend/NOTES.md`.
- Live request/response evidence in the Assignment 4, 5 and 6 transcript files.

Limitations to explain during demonstration

- The File Service records metadata only; it does not upload or persist file bytes.
- The metadata/idempotency store and rate limiter are in memory and reset with the process.
- The Authorization check verifies header presence only; it does not authenticate tokens.
- The scanner partner is a design-only SOAP example; no external scanner is running.
- Demo CORS is permissive for coursework and should be restricted in production.

Before handing in

> **1.** Confirm these four team members and roll numbers are correct.
> **2.** Upload the demonstration to Google Drive.
> **3.** Set link access to **Anyone with the link - Viewer**.
> **4.** Replace the Google Drive placeholder on page 1 with the shareable URL.

GitHub: github.com/si-rish/SecureCloud